LETS: Lineage Escrow Transition Systems for Partition-Safe Autonomous Agent Populations

Anonymous Author(s)
Affiliation withheld in this research draft

Extended arXiv-style manuscript — 8 August 2026

Abstract

Autonomous systems increasingly create descendants, migrate work, and continue operating while disconnected. Existing work separately provides capability delegation, agent lifecycles and contracts, complete mediation, stateful authorization, numeric escrow, bounded counters, leases, hierarchical state machines, and lineage records. These mechanisms do not by themselves specify how a finite, population-wide envelope on protected effects is preserved across a recursively changing agent lineage during network partitions without making every ephemeral agent a distributed-state replica. We propose *Lineage Escrow Transition Systems* (LETS), a reasoner-independent abstract object and enforcement architecture. A bounded set of stable *wardens* owns distributed escrow state; dynamic agents receive signed, expiring, capability-attenuated leases. Every protected effect is an HSM transition with a nonnegative cost vector, and sensor evidence may satisfy a guard but cannot mint authority. We formalize conservation, partition safety, capability attenuation, compact online metadata, and a residual/time bound on disconnected branch-revocation exposure. We also state an availability–revocation impossibility result for disconnected subjects. A small Python kernel passes eight unit tests, and an exhaustive finite-state checker explores 35,209 states and 127,480 transitions without finding a conservation violation. Preliminary simulations illustrate the expected distinction between centralized safety, unsafe eventual accounting, and partition-safe local escrow. These results establish feasibility, not a complete distributed evaluation. The primary research claim is the lineage-oriented transition-escrow object and its operational semantics, not any constituent mechanism.

Claim-status legend. Established.statements summarize established literature. **Assumption.**statements delimit the engineering or threat model. **Proposed.**statements are original claims to be tested. **Preliminary.**statements report results from the included reference artifact. **Future work.**statements identify work not implemented or validated in this manuscript. This distinction is used throughout because the manuscript is an extended research draft rather than a record of a completed deployment.

1 Introduction

Autonomous software is moving from a single decision loop toward populations of processes that divide tasks, create specialized workers, migrate between sites, recover from failure, and invoke increasingly consequential services. The reasoning component may be a large language model (LLM), a symbolic planner, a reinforcement-learning policy, a search procedure, a rules engine, or a robotics controller. The safety problem is not specific to how an action proposal was produced. It arises when a changing set of descendants can cause external effects through tools, networks, storage, sensors, actuators, vehicles, or other agents.

A central authorization service can enforce a global limit but becomes unavailable to disconnected sites. An eventually consistent counter can remain available but permits concurrent partitions to exceed a hard bound. A bounded counter distributes rights safely, but a direct mapping from one dynamic agent to one counter replica couples the coordination structure to potentially unbounded identity churn. Qualitative capabilities restrict which action classes are available but do not bound aggregate quantity. A hierarchical state machine (HSM) constrains legal order but does not automatically limit how many independently executing descendants traverse costly transitions. Agent lifecycle runtimes and resource contracts manage individual or parent–child state, but a recursive population requires explicit semantics for sibling concurrency, partitions, reclamation, migration, and branch revocation.

This manuscript asks a narrower question than "Can an AI self-replicate?" Replication is treated as one lifecycle transition among many:

> *How can a distributed autonomous population remain locally useful during disconnection while a finite, population-wide envelope on protected effects is preserved across every descendant, and while offline revocation exposure and online safety metadata remain bounded?*

We propose LETS, short for *Lineage Escrow Transition Systems*. LETS separates two roles that are often conflated. A small, configured set of stable *wardens* are the distributed escrow replicas and trusted effect mediators. Dynamic agents are *subjects*: they receive signed leases, propose transitions, and may recursively create descendants by partitioning their existing residual rights. The descendant is not added as a permanent replica of a global counter. Each protected transition has an HSM source and target, a required capability, an evidence predicate, and a nonnegative cost vector. The warden atomically validates and debits that transition before a protected executor accepts the effect.

1.1 Contributions and non-claims

Established. Numeric escrow, bounded counters, distributed quota protocols, exactly-once quantity transfer, capability attenuation, recursive delegation, leases, statecharts, runtime enforcement, agent lineage, and agent lifecycle management all predate this work [2, 5, 8, 10, 13, 21, 33, 56, 68, 70, 81, 83]. We do not claim any of these primitives.

Proposed. The intended contributions are:

1. a lineage-oriented *transition escrow* abstract object that composes quantitative rights, qualitative capabilities, HSM legality, nested leases, and branch epochs for recursively changing agent populations;
2. a stable-warden/ephemeral-subject separation that permits local spawn and protected execution during partitions without assigning every historical descendant a distributed-counter replica identity;
3. formal safety properties: vector conservation, partition safety, capability attenuation, nested expiry, compact online safety-state bounds, and bounded disconnected branch-revocation exposure;
4. an impossibility statement clarifying why immediate revocation and unrestricted disconnected availability cannot both be guaranteed without an online oracle;
5. an executable reference kernel, bounded exhaustive checker, TLA+ specification, and reproducible preliminary experiments; and

6. an implementable evaluation plan spanning synthetic lineage stress, sensor-driven action, multiple reasoning backends, adversarial schedules, baselines, and ablations.

Preliminary. The included kernel is not a production authorization service. It implements local recursive spawn, HSM/evidence/capability checks, nested expiry, branch revocation, expiry reclamation, and idempotent free-pool transfer. It does not implement Byzantine wardens or safe active-subtree migration. The latter appears in the proposed protocol but is clearly separated from the implemented v0 path.

1.2 Why this is a multi-agent systems contribution

The framework does not depend on LLM behavior. An LLM can be placed behind the same proposal interface as an HTN planner, a fixed finite-state controller, or a learned policy. The claimed invariant depends on complete mediation by wardens and protected executors, not on prompt compliance. The natural venue is therefore engineering and analysis of multi-agent systems rather than a learning track. The evaluation must nevertheless demonstrate that the abstraction is useful for actual autonomous workflows and not merely a repackaged storage quota.

2 Related Work and Novelty Boundary

The literature review was conducted as a novelty-disproof exercise. A companion literature matrix maps 84 primary sources to their established contribution, relationship to LETS, remaining gap, and audit verdict. This section emphasizes the closest collisions; Section D provides a broader map.

2.1 Autonomous agents, planning, and recursive decomposition

Classical agent theory characterizes autonomy, reactivity, proactivity, and social behavior [77]. Contract Net established distributed manager–contractor task allocation [71]. Hierarchical task-network planning formalized recursive decomposition and expressivity [26], and SHOP2 provided an operational HTN planner [54]. The actor model provides asynchronous local state, messaging, and dynamic process creation [1, 37]. Cooperative multi-agent reinforcement learning, exemplified by QMIX, supplies another decentralized decision backend [63].

Modern LLM-agent methods interleave reasoning and action, branch over candidate thoughts, learn tool use, or revise behavior from feedback [66, 69, 79, 80]. AgentBench and AgentDojo evaluate interactive action and adversarial tool use [20, 49]. These methods affect proposal quality; they do not establish a trusted population-wide effect invariant. In LETS, they are interchangeable reasoners outside the trusted computing base.

2.2 Agent protocols, lifecycle, and runtimes

HAPN uses hierarchical finite-state structures for agent protocols [78]; dynamic protocols support open systems whose interaction rules change [9]. JADE and FIPA define practical agent lifecycle and management services [12, 29]. Borg and Kubernetes demonstrate desired-state orchestration and large-scale process lifecycle management [16, 73].

Two recent works defeat broad versions of our initial proposal. Agent libOS includes long-running agent processes, identity, parent–child lineage, capabilities, checkpoints, and audit [83]. Agent Contracts defines lifecycle states, multidimensional resource contracts, conservation laws, recursive delegation, and dynamic instantiation [81]. Consequently, LETS does not claim novelty for an agent OS, lifecycle state machine, hierarchical budgets, or recursive resource contracts. Its remaining

question is the distributed operational semantics of a lineage-wide hard bound under partitions and churn.

2.3 Agent-specific governance and evolving authority

Faramesh proposes a protocol-agnostic, non-bypassable execution control boundary [28]. AgentBound combines delegation, behavioral governance, site contracts, and receipts [40]. Fixed Ceiling constrains evolving agents under an authority ceiling with attenuating delegation [84]. Provenact coordinates stateful policies and effects to provide serializable governance for concurrent agents [59]. Context Lineage Assurance and Agent Identity URI address provenance and identity [39, 50]. Self-Sovereign Agent examines persistent, economically capable autonomous subjects [62].

These systems make generic claims about standing delegation, policy state, identity, or an authorized subject's evolving authority difficult to defend as novel. The distinction sought by LETS is a finite vector that is *partitioned among simultaneously acting siblings and descendants*, with a stable escrow-replica set, local subleasing, and a quantified offline revocation bound. This is a synthesis claim and therefore must be demonstrated by a precise object and discriminating baselines, not by architectural vocabulary alone.

2.4 Escrow, quotas, counters, and recursive delegation

The escrow transactional method partitions rights so local operations preserve numeric constraints [56]. Distributed quota enforcement has been studied for spam, storage, cloud infrastructure, and tree organizations [11, 43, 61, 75]. Bounded CRDTs enforce numeric invariants without coordination on every operation [10]; scalable eventually consistent counters and dynamic identity work address unreliable networks and replica identities [5, 25]. Exactly-once quantity transfer prevents duplicate rights movement [70].

Most importantly, Agudo, Fernandez-Gago, and Lopez present quota-based recursive delegation over finite resources [2]. We therefore explicitly disclaim recursive quota delegation as a contribution. LETS asks whether a quota object can be reified as protected HSM transitions over a changing lineage, implemented with stable wardens and ephemeral leases, and paired with branch-scoped revocation and metadata bounds. A reviewer may still conclude that this composition is obvious; the evaluation in Section 9 is designed to make that judgment falsifiable.

2.5 Capabilities, delegation, and revocation

Capability systems provide unforgeable authority references [21]; complete mediation and least privilege are canonical protection principles [65]. Macaroons add contextual caveats and attenuation [13]. WAVE provides decentralized authorization with transitive delegation [8]. OASIS studies access control in open distributed environments [35]. Chuat et al. survey persistent delegation and revocation weaknesses [17]. Proof-carrying code and verified kernels reduce trust in untrusted components through proofs or verified enforcement [53, 55].

Capabilities answer "may this subject perform this class of action?" A conserved vector additionally answers "how much of the finite population envelope remains?" Neither should substitute for the other. LETS enforces capability subset attenuation on spawn and vector subtraction on allocation and execution.

2.6 State machines, formal methods, and runtime assurance

Statecharts and STATEMATE define hierarchical and concurrent state-machine semantics [33, 34]. Petri nets model concurrency and token flow [52]. Temporal logic and TLA support safety and liveness reasoning [44, 60]; model checking explores finite-state concurrent systems [18]; timed automata model clocks [6]. Bigraphical reactive systems formalize changing topology [51]. Security automata, shield synthesis, and online verification constrain unsafe execution at runtime [14, 58, 68].

An HSM alone constrains order but not aggregate descendant consumption. A Petri net could express conservation, but it does not by itself supply signed identity, partition protocol, complete mediation, or deployment semantics. LETS uses established formal tools to specify a new candidate object; the formalism itself is not the contribution.

2.7 Cyber-physical systems, sensors, swarms, and self-management

Cyber-physical systems couple computation to physical processes [46]. Sensor-network research studies scalable coordination, energy-aware communication, and constrained runtimes [4, 27, 36, 38]. Swarm behavior and swarm robotics demonstrate decentralized collective action [15, 23, 41, 64, 76]. Self-stabilizing, autonomic, and architecture-based self-adaptive systems address recovery and adaptation [22, 30, 42].

These fields motivate the workload and failure model. Sensor observations are not authority in LETS: they can satisfy an evidence guard, but only preallocated rights can authorize a protected effect. Repair and replication likewise transfer or consume existing rights rather than creating net authority.

2.8 Replication, self-modification, and AI safety

Self-reproduction has long been studied in automata and artificial life [45, 74]; computer viruses instantiate adversarial replication [19]; biological and evolutionary models study replication and adaptation [24, 72]. Gödel machines formalize self-referential improvement [67]. Recent studies evaluate self-replication risk, autonomous hacking, adaptive worms, and safety amplification in self-evolving agents [3, 31, 48, 82].

AI safety work identifies side effects, unsafe exploration, reward gaming, and interruptibility [7, 32, 47]. Multi-Agent Security Tax quantifies a collaboration–security trade-off under compromised agents [57]. LETS treats spawn as an ordinary protected transition and reports a safety–availability frontier rather than assuming replication is uniquely special.

2.9 Closest-work comparison

3 Background and Design Principles

3.1 Why a vector rather than one counter

Autonomous systems consume heterogeneous resources: actuator commands, network egress, energy, money, data access, compute, hazardous operations, or domain-specific risk points. A scalar may be adequate for one policy, but a vector makes independent ceilings explicit. LETS assumes a fixed dimension m per root policy and uses componentwise order. For $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{N}^m$, write $\boldsymbol{x} \preceq \boldsymbol{y}$ when $x_j \leq y_j$ for every dimension j.

Vectors do not solve semantic commensurability. A policy author must choose dimensions and transition costs. A "risk point" is an engineering encoding, not a physical law. The formal result is conditional on that encoding and on complete mediation of the effects it is intended to cover.

Table 1: Closest prior work and the narrow remaining claim. "Missing" means outside the cited work's principal model, not necessarily impossible to add.

Work	Established contribution relevant to LETS	Remaining distinction claimed by LETS
Agent Contracts [81]	Multidimensional budgets, lifecycle states, conservation, recursive delegation, dynamic agents	Partition-safe distributed object; stable wardens versus ephemeral subjects; branch exposure bound
Agent libOS [83]	Runtime, lineage, identity, capabilities, checkpointing, audit	Population-wide conserved effect vector under disconnection
Provenact [59]	Serializable policy state and coordinated state/effect enforcement	Offline recursive subleasing and bounded local rights across descendants
Fixed Ceiling [84]	Authority ceiling and attenuating delegation across evolving agents	Simultaneous allocation among sibling descendants and cross-site escrow
Quota delegation [2]	Recursive delegation over finite resources	HSM effect semantics, stable-warden split, partition protocol, branch revocation bound
Bounded CRDTs [10]	Partition-safe numeric invariants	Dynamic lineage subject semantics without one replica per descendant
Delegated credentials [8, 13]	Transitive and attenuated qualitative authority	Quantitative conservation and state-transition legality
Faramesh [28]	Non-bypassable action authorization boundary	Distributed finite rights, local subleasing, and partition availability
Tree/cloud quotas [11, 43]	Decentralized and hierarchical global quotas	Agent lineage, capabilities, evidence, HSMs, and branch-scoped offline exposure

3.2 Why transitions rather than arbitrary API calls

A raw quota answers whether a quantity remains. It cannot prevent an action that is quantitatively cheap but procedurally illegal, such as actuating before validation, ending a protocol before it begins, or releasing a resource before settlement. An HSM transition provides an explicit source, target, capability, evidence predicate, and cost. A protected effect is authorized only as part of that transition.

3.3 Why stable wardens

Dynamic membership is costly when every subject participates in distributed invariant maintenance. LETS instead configures a bounded warden set W. Wardens can be replicated and rebalanced, but their identity is operational infrastructure rather than agent-created population state. Agents are short-lived lease holders. This decision narrows the trust and decentralization claim, but it makes online safety-state growth depend on active leases rather than every historical agent identity.

3.4 Why finite leases

A disconnected agent cannot receive a revocation message. A finite lease creates a maximum offline authority horizon. Descendant expiry is nested beneath ancestor expiry, preventing a child from extending authority beyond its grant. Short leases reduce exposure and availability; this trade-off is a primary evaluation target, not an implementation detail.

4 Problem Definition

4.1 System entities

Let $W = \{w_1, \ldots, w_n\}$ be a finite configured set of wardens. Let $A(t)$ be the dynamic set of agent subjects at time t. Agents form a rooted lineage forest $G_t = (A(t), E_t)$ in which each non-root agent has one authorization parent. The execution topology may differ from the authorization tree; migration changes location but not lineage.

A machine specification is

$$M = (S, s_0, T),$$

where S is a finite set of states, $s_0 \in S$, and each transition

$$\tau = (s_{\mathrm{src}}, s_{\mathrm{dst}}, k, g, \boldsymbol{c}) \in T$$

has a required capability k, evidence guard g, and nonnegative cost $\boldsymbol{c} \in \mathbb{N}^m$.

Definition 1 (Lease)**.** *A lease ℓ is a record*

$$\langle id, lin, parent, subj, warden, \boldsymbol{a}, \boldsymbol{r}, K, h_M, s, path, e, t_i, t_x, status, seq, \sigma \rangle,$$

where $\boldsymbol{a}$ is the original allocation, $\boldsymbol{r}$ is authoritative residual rights, K is a capability set, h_M is a machine digest, s is current state, e is a branch epoch, $[t_i, t_x)$ is the validity interval, and σ authenticates immutable grant fields. The mutable residual and state are held authoritatively by the warden and are bound to monotonic receipts.

4.2 Threat and failure model

Assumption 1 (Untrusted reasoner)**.** *The reasoner, generated code, task memory, and agent runtime may be buggy or adversarial. They may propose illegal transitions, replay requests, alter tokens, attempt capability amplification, or overspend.*

Assumption 2 (Trusted wardens and executors)**.** *Wardens, their durable state, signing keys, and protected executor receipt checks are trusted. Wardens may crash and restart but are not Byzantine in the main model. Every effect inside the policy scope passes through a receipt-enforcing executor.*

Assumption 3 (Network)**.** *Messages may be delayed, duplicated, reordered, dropped, or partitioned for an unbounded interval. Local durable transactions are atomic. Cross-warden transfer uses idempotent messages.*

Assumption 4 (Time)**.** *Each warden has a monotonic clock whose real-time uncertainty is bounded by ϵ for claims involving expiry. If this bound is violated, time-based exposure claims do not hold, although vector conservation still does.*

Assumption 5 (Evidence)**.** *Evidence adapters may be compromised unless separately authenticated. An evidence predicate can deny or enable a transition; it cannot increase $\boldsymbol{r}$ or K. The framework does not prove that a physical observation is true.*

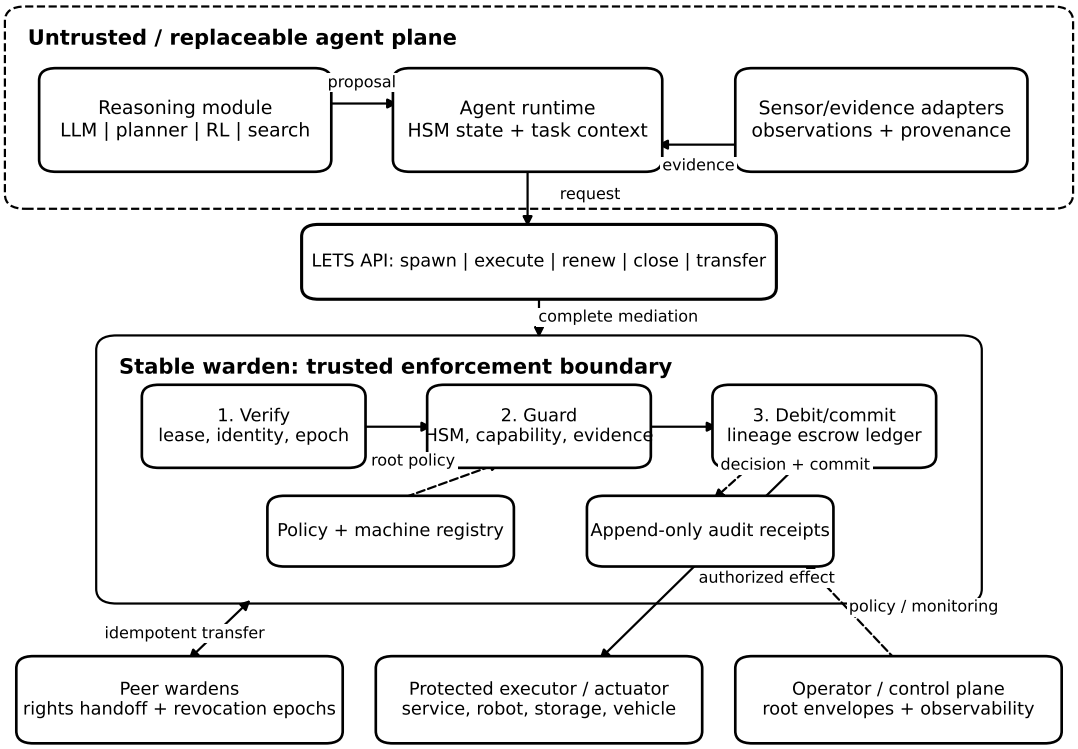


Figure 1: LETS architecture. The reasoner and runtime are replaceable and untrusted. The stable warden is the trusted enforcement boundary. Sensor evidence can satisfy a guard but cannot mint rights.

4.3 Goals and non-goals

The goals are: (G1) componentwise population safety; (G2) local progress during partition when local rights exist; (G3) capability attenuation; (G4) bounded disconnected revocation exposure; (G5) online safety-state growth independent of historical descendants after compaction; and (G6) reasoner independence.

Non-goals are Byzantine-warden tolerance, protection against a bypassable actuator, automatic derivation of correct transition costs, perfect sensor truth, confidentiality of lineage metadata, incentive-compatible economics, and unrestricted active-subtree migration in v0.

5 Lineage Escrow Transition Systems

5.1 Architecture

Figure 1 separates the untrusted agent plane from a stable warden and a protected executor. A reasoner proposes a transition; evidence adapters provide observations and provenance; the warden verifies the lease and branch epoch, resolves the HSM transition, checks capability and evidence, debits residual rights, updates state, and emits a receipt. Peer wardens exchange rights and revocation epochs. The executor accepts only a valid receipt bound to the request.

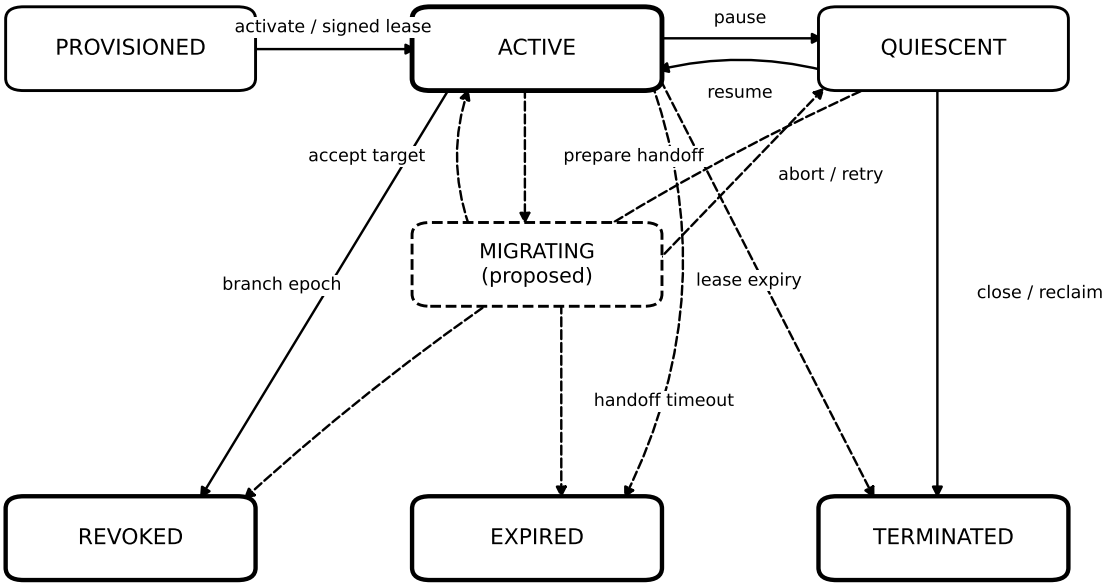


Figure 2: Lease lifecycle. Dashed migration edges are proposed. Terminal records reject protected transitions; residual rights are not silently duplicated or discarded.

5.2 Lifecycle

Figure 2 distinguishes active, quiescent, and terminal lease states. Revoked residual remains conserved but unusable until closure or policy-authorized reclamation. Descendant expiry is nested beneath parent expiry. The proposed migrating state requires a single-authority ownership handoff and is not implemented by the current reference kernel.

5.3 Root issuance

An operator creates a root envelope $\boldsymbol{b}_q \preceq \boldsymbol{B}$ for policy q. The selected warden subtracts the allocation from its free pool and creates a signed root lease. Root issuance is an administrative operation; if multiple control-plane actors can issue roots, their aggregate issuance must itself be mediated by the same global escrow.

5.4 Recursive local spawn

A parent can create a child locally when:

$$\boldsymbol{a}_{child} \preceq \boldsymbol{r}_{parent}, \quad K_{child} \subseteq K_{parent}, \quad t_{x,child} \leq t_{x,parent}.$$

The warden atomically sets

$$\boldsymbol{r}'_{parent} = \boldsymbol{r}_{parent} - \boldsymbol{a}_{child}, \qquad \boldsymbol{r}_{child} = \boldsymbol{a}_{child}.$$

No global membership update is required. The parent and child can then independently consume their disjoint residual vectors.

5.5 Protected execution

For a request $q = (id_\ell, \tau, evidence, nonce)$, a warden performs:

Listing 1: Abstract protected-transition operation.

```
EXECUTE(q, now):
  l <- durable_lookup(q.lease_id)
  require l.status == ACTIVE and now < l.expiry
  require branch_epoch_is_current(l)
  require authenticate_subject(l, q) and verify_grant(l)
  tau <- machine[l.machine_digest].resolve(l.state, q.transition)
  require tau.capability in l.capabilities
  require tau.guard(q.evidence) == true
  require tau.cost <= l.residual                  # componentwise
  atomically:
      l.residual <- l.residual - tau.cost
      consumed   <- consumed + tau.cost
      l.state    <- tau.target
      l.sequence <- l.sequence + 1
      receipt    <- sign(executor, l, tau, evidence_digest, nonce)
      append_audit(receipt)
  return receipt
```

The executor verifies the receipt's signature, audience, nonce, expiry, and monotonic sequence before causing the external effect. This "debit before effect" order prevents a successful effect from escaping accounting, but it can produce a debit without an effect if the executor fails after authorization. Applications may represent refundable reservations separately or use a commit protocol. LETS v0 models nonrefundable protected attempts; exact business semantics are domain specific.

5.6 Close, expiry, and reclamation

A close operation marks the lease terminal and returns its remaining residual to a designated warden pool exactly once. Expiry prevents further action. Reclamation after expiry is safe only under the clock and complete-mediation assumptions: an expired subject cannot still reach an executor that accepts its old receipt. A conservative implementation may quarantine residual until an additional grace interval passes.

5.7 Branch revocation

A revocation message identifies a lineage branch root and a monotonically increasing epoch. Connected wardens mark matching leases revoked and reject renewal. A descendant carries a branch path or compact proof sufficient to match revocation prefixes. The path representation in the reference kernel is simple and linear in depth; production systems may use Merkle or interval labels, with different update and privacy costs.

5.8 Cross-warden rights transfer

Free rights are rebalanced using an idempotent handoff. Preparing transfer x from w_s to w_t removes $\boldsymbol{v}_x$ from the source free pool and places it in an in-flight set. The target accepts a source sequence at most once, moves $\boldsymbol{v}_x$ into its free pool, and acknowledges. Duplicate accepts are no-ops. Online duplicate state can be compacted with a contiguous per-peer high-water mark and a bounded sparse gap set rather than retaining every transfer identifier.

5.9 Active-subtree migration

Future work.A correct active-subtree migration must ensure that one warden is authoritative for each mutable lease at a time. One design is: quiesce the subtree, freeze residual and sequence ranges at the source, prepare a signed ownership-epoch token, accept and install at the target exactly once, redirect the executor audience, then finalize the source. Timeouts either abort before target acceptance or leave the source frozen until recovery resolves ownership. This protocol is shown in Figure 3 for completeness but is not implemented or experimentally claimed in v0.

6 Formal Model

6.1 State

Let m be the number of resource dimensions and $\boldsymbol{B} \in \mathbb{N}^m$ the configured total envelope. For each warden $w \in W$, $\boldsymbol{f}_w \in \mathbb{N}^m$ is free escrow. Let L include every lease whose residual has not been returned, including revoked or expired records pending reclamation. Each $\ell \in L$ has residual $\boldsymbol{r}_\ell \in \mathbb{N}^m$. Let $\boldsymbol{C} \in \mathbb{N}^m$ be cumulative consumed rights. Let X be prepared, unaccepted transfers and $\boldsymbol{x}_z$ the amount of transfer $z \in X$.

Definition 2 (Conservation invariant)**.** *The system is conserved when, componentwise,*

$$\boldsymbol{B} = \boldsymbol{C} + \sum_{w \in W} \boldsymbol{f}_w + \sum_{\ell \in L} \boldsymbol{r}_\ell + \sum_{z \in X} \boldsymbol{x}_z. \tag{1}$$

The invariant accounts for ownership, not utility. Consumed rights remain in $\boldsymbol{C}$ and cannot be reused unless a separate policy explicitly models refundable reservations.

6.2 Operations

The abstract kernel has the following state transformations:

IssueRoot$(w, \boldsymbol{v})$
: Require $\boldsymbol{v} \preceq \boldsymbol{f}_w$; subtract $\boldsymbol{v}$ from $\boldsymbol{f}_w$ and create root residual $\boldsymbol{v}$.

Spawn$(p, c, \boldsymbol{v})$
: Require $\boldsymbol{v} \preceq \boldsymbol{r}_p$ plus capability and expiry attenuation; subtract from p and create child c with $\boldsymbol{v}$.

Execute(ℓ, τ)
: Require enabled transition and $\boldsymbol{c}(\tau) \preceq \boldsymbol{r}_\ell$; subtract cost and add it to $\boldsymbol{C}$.

Close/Reclaim(ℓ, w)
: Remove residual $\boldsymbol{r}_\ell$ from the lease term and add it to $\boldsymbol{f}_w$ exactly once.

Prepare$(s, t, \boldsymbol{v})$
Subtract $\boldsymbol{v}$ from $\boldsymbol{f}_s$ and add an in-flight transfer of $\boldsymbol{v}$.

Accept(z) Remove $\boldsymbol{x}_z$ from the in-flight term and add it to the target free pool exactly once.

Revoke(b, e) Change usability/status and epoch state but do not move or create rights.

6.3 Conservation and partition safety

Theorem 1 (Vector conservation)**.** *If Equation (1) holds initially and every state-changing operation is one of the abstract operations above executed atomically at its owner, then Equation (1) holds after every finite execution trace.*

Proof. By induction on trace length. The base case is initialization, which partitions $\boldsymbol{B}$ among warden free pools. For the inductive step, IssueRoot moves $\boldsymbol{v}$ from one free-pool summand to a lease summand; Spawn moves $\boldsymbol{v}$ from a parent residual summand to a child residual summand; Execute moves $\boldsymbol{c}(\tau)$ from a lease residual to $\boldsymbol{C}$; Close/Reclaim moves a residual from a lease to a free pool; Prepare moves $\boldsymbol{v}$ from a free pool to the in-flight sum; Accept moves it from the in-flight sum to the target free pool. Revoke changes no vector. Each operation preserves the componentwise sum. Exactly-once target acceptance is necessary for the Accept case. □

Corollary 1 (Population bound under partition)**.** *Under the assumptions of Theorem 1, for every dimension j and every message delay or network partition, $C_j \le B_j$.*

Proof. Every other term in Equation (1) is nonnegative. Network partition can prevent transfer or renewal but cannot duplicate an already disjoint local residual. Therefore $C_j = B_j - (\sum f_{w,j} + \sum r_{\ell,j} + \sum x_{z,j}) \le B_j$. □

This is a safety result, not a liveness result. A site with insufficient local rights can deny work even when unused rights exist elsewhere.

6.4 Capability attenuation and nested expiry

Theorem 2 (Lineage attenuation)**.** *If root capabilities are operator-issued, Spawn requires $K_c \subseteq K_p$, and no other operation adds capabilities, then for every descendant d of root r, $K_d \subseteq K_r$.*

Proof. Induct on lineage depth and apply transitivity of set inclusion. □

Theorem 3 (Nested authority horizon)**.** *If every child issue and renewal satisfies $t_{x,c} \le t_{x,p}$, then no descendant lease remains valid after any ancestor's expiry.*

Proof. Induct along the ancestor path. The minimum ancestor expiry is an upper bound on each descendant expiry. □

6.5 Transition soundness

Theorem 4 (Mediated transition legality)**.** *Assume every policy-scoped effect is accepted only with a fresh warden receipt and a warden emits a receipt only after executing Listing 1. Then every accepted effect corresponds to an HSM transition enabled from the authoritative source state, with a satisfied capability requirement, evidence predicate, and residual-cost check.*

Proof. Immediate from complete mediation and the receipt-issuance preconditions. The theorem does not establish that the HSM or evidence policy is semantically correct. □

6.6 Online metadata bound

Let $n = |W|$, L_a be active or unreclaimed leases, and R_a the size of the active revocation representation. A bounded-counter implementation over stable wardens uses $O(mn^2)$ rights-transfer state in a matrix formulation. Per-live-lease state is $O(m)$ plus identity, state, and lineage proof. Sequence-watermark duplicate suppression is $O(n^2 + G)$ for bounded gap windows G.

Theorem 5 (Online safety-state bound)**.** *After inactive lease records and accepted-transfer identifiers are safely compacted, online safety metadata is*

$$O(mn^2 + mL_a + R_a + G),$$

independent of the number H of historical descendants.

Proof. The retained terms are the stable warden transfer state, live/unreclaimed lease vectors, current revocation representation, and bounded transfer gaps. Historical leases and contiguous accepted transfers are not needed to decide new transitions after their residual has been returned and their replay window has closed. Append-only audit storage may remain $O(H)$; it is archival, not online invariant state. □

The reference kernel does not implement the complete compaction protocol: it stores accepted transfer identifiers in a set. Theorem 5 applies to the proposed sequence-watermark implementation and is an engineering obligation for the full prototype.

6.7 Bounded offline revocation exposure

Suppose branch b is revoked at real time t_r. Let $R_{b,j}(t_r)$ be all usable residual rights in dimension j held by disconnected descendants at that instant. Let $T_b(t_r)$ be the maximum real time until every such nested lease is rejected by expiry, including clock uncertainty. Let ρ_j be an enforceable upper bound on dimension-j consumption rate, if such a bound exists.

Theorem 6 (Residual/time exposure bound)**.** *Under complete mediation, nested expiry, and clock uncertainty at most ϵ, post-revocation branch consumption in dimension j satisfies*

$$E_{b,j} \le \min\left(R_{b,j}(t_r),\ \rho_j\left(T_b(t_r) + \epsilon\right)\right). \tag{2}$$

If no rate bound is enforced, the residual bound $E_{b,j} \le R_{b,j}(t_r)$ still holds.

Proof. Conservation and absence of renewal after the branch learns the epoch prevent the disconnected branch from consuming more than its residual. Nested expiry and clock uncertainty bound the period during which receipts can be emitted. Under a rate cap, consumption over that interval is at most rate times duration. Both bounds apply, so their minimum applies. □

The theorem is intentionally modest. It does not claim that a disconnected branch stops immediately. It turns the unavoidable residual authority into a configured and measured exposure.

6.8 Availability–revocation impossibility

Proposition 1 (No immediate offline revocation with unrestricted availability)**.** *In an asynchronous system, a subject isolated from all trusted revocation information cannot both (i) remain authorized to perform an unbounded sequence of effects during disconnection and (ii) be guaranteed to stop immediately after a remote revocation event.*

Proof. Consider two executions indistinguishable to the isolated subject and its local mediator up to the next effect: in one, no revocation occurs; in the other, a remote revocation occurs after partition. If the local system authorizes the effect to preserve unrestricted availability in the first execution, it makes the same decision in the indistinguishable revoked execution, violating immediate revocation. If it rejects, unrestricted availability fails in the non-revoked execution. An online oracle, bounded lease, preallocated finite residual, or stronger synchrony assumption is required to distinguish or bound the executions. □

7 Protocol and API Design

7.1 Request and receipt schema

A concrete API can expose the following endpoints or RPCs:

Listing 2: Conceptual mediation API.

```
POST /v1/roots
POST /v1/leases/{parent}/children
POST /v1/leases/{lease}/transitions
POST /v1/leases/{lease}/renew
POST /v1/leases/{lease}/quiesce
POST /v1/leases/{lease}/resume
POST /v1/leases/{lease}/close
POST /v1/branches/{lease}/revoke
POST /v1/transfers/prepare
POST /v1/transfers/{sequence}/accept
GET  /v1/leases/{lease}
GET  /v1/invariants
```

An execution request binds a request identifier, subject authentication, lease identifier, transition name, evidence references or digest, executor audience, and nonce. A receipt binds warden identity, policy and machine versions, lease and lineage identifiers, source/target state, transition cost, resulting sequence, evidence digest, executor audience, nonce, and short expiry. The protected executor should persist used nonces or a monotonic sequence window.

7.2 Atomic storage transaction

The debit, HSM update, sequence increment, and durable receipt record must commit atomically. A PostgreSQL implementation can lock the lease row and update a warden aggregate in one transaction. A RocksDB or SQLite implementation can use a write batch or immediate transaction. Sending the external effect is outside this transaction; applications must choose at-most-once, at-least-once, or reservation/commit semantics. The paper's core conservation theorem covers rights, not exactly-once physical effects.

7.3 Scheduling and rights rebalancing

Within a warden, fair scheduling can prevent one lineage from monopolizing CPU even if it owns residual rights. Deficit round robin is a simple default. Across wardens, a rebalancer estimates demand and performs quantity transfers while retaining a local reserve. Static equal partitioning maximizes predictability; adaptive rebalancing improves utilization but adds coordination and prediction error. Both preserve safety when transfer ownership is exact.

7.4 Failure handling

Warden crash. Durable local transactions restore authoritative residual and sequence. In-flight executor receipts may be replayed; the executor's nonce window prevents duplicate effects.

Duplicate or reordered transfer. Target acceptance is idempotent by source/target/sequence. Out-of-order sequences enter a bounded gap set. The source retains an in-flight record until acknowledged or administratively reconciled.

Parent crash. Children already own disjoint residual and can continue. No rights are reclaimed merely because the parent process disappears. Parent lease expiry or explicit close determines reclamation.

Partition healing. Wardens exchange branch epochs, transfer acknowledgments, and audit digests. Local consumption needs no merge arithmetic beyond authoritative local state because the rights were disjoint before partition.

Clock uncertainty. If clock health exceeds the configured uncertainty bound, a conservative warden should stop renewal and possibly protected execution rather than silently extend the revocation horizon.

8 Implementation

8.1 Reference kernel

Preliminary. The included Python 3 kernel is intentionally small and auditable. Resource vectors are immutable tuples. A `MachineSpec` hashes transition metadata. A `Lease` stores allocation, residual, capabilities, machine digest, state, ancestor path, timestamps, epoch, status, and an Ed25519 signature over immutable grant fields. A `Warden` implements issue, local spawn, execute, quiesce/resume, nested renewal, branch revocation, close, and expiry reclamation. A `LETSystem` partitions an initial budget among wardens and implements prepare/accept transfer.

The mutable residual and current state are not trusted from a bearer token; they are authoritative in the warden object. Spawn subtracts the child's allocation before issuing the child. Execute verifies the grant signature, HSM state, capability, evidence predicate, and residual. The child expiry is capped by parent expiry at issuance and renewal. Revoked residual remains conserved until close or expiry reclamation.

8.2 Tests and exhaustive checker

Preliminary. Eight unit tests cover recursive conservation, capability amplification rejection, nested descendant expiry, state and evidence guards, branch revocation, idempotent expiry reclamation, signature tampering, and duplicate transfer acceptance. An independent exhaustive checker explores a scalar finite model with budget 4, four lease identifiers, bounded depth 10, issue/spawn/consume/-close/prepare/accept operations, and duplicate accept self-loops. It reports 35,209 unique states, 127,480 transitions, 8,986 duplicate-accept self-loops, and no conservation violation.

These checks do not prove the full distributed implementation. The finite checker abstracts cryptography, time, capabilities, evidence, crash persistence, and active migration. The TLA+

artifact specifies the conservation kernel but is not claimed here as an executed TLC result. A submission version should run TLC or Apalache and archive the model-check configuration and logs.

8.3 Deployment architecture

Figure 4 shows the intended testbed: two edge sites with local sensors, ephemeral agents, durable wardens, and receipt-enforcing services, plus policy, metrics/fault injection, a third rebalancing warden, and an audit sink. A site continues protected transitions during partition until its local vector or lease lifetime is exhausted.

8.4 Full prototype obligations

Future work.A publication-ready implementation must add durable transactions, authenticated network APIs, executor receipt verification and replay state, sequence-watermark transfer compaction, crash recovery, fault injection, trace export, and at least three comparable baselines. Safe active-subtree migration should either be completed and tested or removed from the main claim.

9 Experimental Methodology

9.1 Research questions and hypotheses

The evaluation is organized around six questions: invariant preservation (RQ1), partition availability (RQ2), revocation exposure (RQ3), metadata scaling (RQ4), reasoner independence (RQ5), and utility cost (RQ6). The primary outcomes are global overrun, completed safe work, post-revocation consumption, p99 transition latency, and online safety-state bytes. These outcomes should be declared before the full campaign.

9.2 Baselines

1. **Online central coordinator:** every protected operation contacts one coordinator; safe but unavailable to disconnected sites.
2. **Consensus-backed coordinator:** a Raft service remains safe and available to a quorum but denies minority partitions.
3. **Naive eventual counter:** each site spends from a stale shared value and merges later; available but can overrun.
4. **Replica-per-agent bounded counter:** each dynamic agent is represented as a rights replica; expected to be safe but metadata/reconfiguration-sensitive.
5. **Static tree quota:** rights are prepartitioned along a fixed hierarchy; safe but potentially underutilized under changing demand.
6. **Parent-local contract:** recursive parent–child budgets at one runtime boundary without cross-warden escrow.

The baseline implementation must use the same workload generator, persistence level, cryptographic path where applicable, and executor semantics. A bare in-process integer is only a diagnostic microbenchmark, not a fair systems baseline.

9.3 Workloads

Synthetic lineage stress. Parameters include branching factor, depth, churn, active population, transition-cost distribution, spatial demand skew, and partition schedule. The generator should include noncommutative HSM paths so a counter-only ablation can fail procedurally.

Sensor-response workflow. MQTT events trigger observe–validate–act transitions. Evidence includes timestamp, source identity, sequence, signature, and payload digest. Faults include stale, forged, missing, and conflicting observations. The protected executor simulates or controls an actuator.

Distributed logistics. Agents assign tasks and vehicles across sites under energy, communication, and action ceilings. Partitions produce demand imbalance and expose rights-rebalancing trade-offs.

Reasoner swap. A fixed policy, symbolic planner, learned policy if feasible, and optional LLM proposer use the same machine and warden API. Safety should remain invariant while task utility changes.

9.4 Fault and attack injection

The harness should delay, duplicate, reorder, and drop messages; create burst and long-lived partitions; kill/restart wardens; replay receipts; revoke branches during disconnection; skew clocks to and beyond the assumed bound; kill parents during spawn; create deep/hot branches; forge evidence; alter lease fields; request capability amplification; and exhaust one vector dimension while others remain.

9.5 Metrics

Safety metrics are vector overrun, invariant violations, illegal transition receipts, capability amplification, duplicate transfer acceptance, and replayed effects. Utility metrics are completed work, deadline success, partition availability, fairness, and false rejection. Systems metrics are throughput, p50/p95/p99 latency, durable transaction cost, transfer traffic, recovery time, online metadata, archival log size, and CPU/memory. Revocation metrics are post-revocation actions, vector consumption, and time to quiescence or expiry.

9.6 Ablations

Ablations remove or alter: lease expiry, nested expiry, HSM guards, capability attenuation, idempotent transfer, stable wardens (all agents become replicas), adaptive rebalancing, evidence freshness, signature verification caching, and branch prefix representation. Each ablation should be linked to a specific theorem or engineering claim.

9.7 Statistical analysis

Use at least 30 independent seeds for stochastic conditions, report means or medians with 95% bootstrap confidence intervals, use Mann–Whitney U or paired Wilcoxon tests as appropriate, report Cliff's delta, and apply Holm correction within each hypothesis family. Deterministic safety failures should be reported as traces and counts rather than converted into weak significance claims. Hardware, software, clock configuration, seeds, and fault profiles must be archived.

Table 2: Estimated effort for the full evaluation.

Task	Effort
Machine-checked model and counterexamples	2–3 days
Persistent multi-warden service	6–8 days
Fault proxy and transfer protocol	4–6 days
Core baselines	5–7 days
Sensor-response workload	5–7 days
Reasoner-swap integration	3–5 days
Full benchmark and statistical analysis	5–7 days
Artifact hardening and clean reproduction	3–4 days

9.8 Engineering effort

Table 2 estimates focused implementation effort after the current kernel. These estimates assume one experienced researcher and are planning values, not measured labor.

10 Preliminary Evaluation

10.1 Status and interpretation

Preliminary. The preliminary experiments are sanity checks. They use a synthetic simulator designed to expose the core trade-off. They should not be read as evidence that the present Python kernel outperforms mature distributed systems. The central baseline intentionally denies disconnected work; the naive eventual baseline intentionally overspends; and some outcome distributions therefore separate completely.

10.2 Partition safety simulation

Thirty seeds simulate an abstract budget of 1,000 protected actions under partitions. LETS begins with disjoint local rights. The central coordinator denies attempts that cannot reach it. The naive eventual counter allows disconnected sites to spend against stale state.

LETS completes exactly 1,000 protected actions per run, with no overrun and no coordination denial; further requests are denied for exhausted local resource. The central coordinator completes a mean 427.77 actions and records a mean 1,497.47 coordination denials. The naive eventual counter completes a mean 1,694 actions but overruns by a mean 694 and violates the invariant in 30/30 runs. Mann–Whitney comparisons produce complete effect separation for several metrics (absolute Cliff's delta 1.0; Holm-adjusted $p \approx 7.2 \times 10^{-12}$), which reflects the constructed mechanisms more than empirical uncertainty.

The result establishes that the harness distinguishes three intended semantics. It does not establish superiority to bounded counters, consensus systems, or optimized quota protocols; those are required full baselines.

10.3 Revocation exposure–availability frontier

A synthetic study varies lease TTL while partitions have mean duration 20 seconds and the action rate is two per second. Figure 7 reports post-revocation actions and disconnected availability. A one-second TTL yields approximately 1.95 post-revocation actions and 0.17 availability; a 60-second

TTL yields approximately 37.9 actions and 0.989 availability. The curve is the expected consequence of finite leases and supports reporting TTL as a policy parameter rather than choosing it silently.

10.4 Metadata scaling analysis

The analytic comparison fixes eight wardens, four resource dimensions, and 1,000 active leases while historical descendants grow from 10^3 to 10^6. The LETS accounting model remains at 5,256 abstract cells because it counts the stable warden matrix plus active lease state. A vector-per-historical-agent representation grows linearly; a matrix-style bounded counter over all historical agents grows quadratically in the illustrative model.

The comparison should be treated cautiously. A real replica-per-agent system would garbage-collect departed replicas and might use sparse representations. The full experiment must implement that comparator fairly and measure both reconfiguration cost and retained metadata.

10.5 Reference microbenchmark

The in-process Python path with Ed25519 verification reaches a median 8,752 transition operations/s over five repetitions of 20,000 operations. A bare in-process centralized integer reaches approximately 30.3 million operations/s. This is not an apples-to-apples comparison: the latter omits signatures, HSM lookup, evidence checks, objects, audit events, persistence, and networking. The only defensible conclusion is that the unoptimized reference kernel is fast enough for experimentation and that cryptographic/interpretive mediation is not free.

11 Discussion

11.1 What LETS adds, if anything

The strongest skeptical interpretation is that LETS is bounded escrow plus leases, capabilities, and state machines. That interpretation is partly correct: every primitive is established. The proposed research value is the lineage-oriented abstract object and the consequences of separating stable replicas from ephemeral subjects. A successful paper must show that a naive composition fails on ownership, revocation, or metadata; that the LETS operation semantics close those gaps; and that the distinction matters in a real autonomous workload.

The work should be rejected if it merely renames a tree quota. The differentiating burden is therefore high. In particular, the final artifact must include: a replica-per-agent bounded-counter baseline; branch revocation under churn; noncommutative HSM attacks; a real receipt-enforcing executor; and safe compaction. Without these, the novelty claim remains plausible but under-supported.

11.2 Why trusted wardens are not hidden decentralization

Wardens deliberately centralize trust into a bounded infrastructure set. LETS is not a permissionless or Byzantine system. The benefit is that dynamic agent creation does not dynamically enlarge the consensus or escrow replica set. This is appropriate for organizations, fleets, industrial sites, healthcare systems, or emergency-response networks that control enforcement infrastructure while distrusting replaceable reasoning modules. It is less appropriate for open adversarial ecosystems without shared wardens.

11.3 State order and quantity are complementary

Consider a workflow with states `provisioned`, `validated`, `active`, and `settled`. A pure counter can prevent more than B actions but may authorize act-before-validate. A pure HSM can ensure order but may allow 10^6 descendants each to execute a legal costly action. LETS's claim is that the conjunction is operationally useful: each effect must be both legal in state and affordable from the conserved lineage vector.

11.4 Sensor evidence is not authority

A sensor may report a fire, hypoxia, equipment failure, traffic obstruction, or scientific observation. Such evidence can legitimately alter which transition is enabled. Letting the observation mint rights would permit a compromised sensor to amplify population authority. In LETS, emergency policy must allocate an emergency reserve in advance or authorize a control-plane root update through a separately protected path.

11.5 Domain transfer

The same object can encode different dimensions and machines:

- robotics: motion, energy, manipulator, and communication budgets;
- aviation: dispatch, route change, maintenance action, and data-access ceilings;
- healthcare: contact, notification, device command, and protected-data access;
- manufacturing: machine starts, material use, hazardous operations, and network egress;
- emergency response: vehicle dispatch, supply allocation, sensor activation, and communications;
- scientific automation: instrument runs, sample consumption, compute, and external service calls.

Transferability does not mean one policy fits all domains. Each deployment needs domain-specific cost semantics, executor coverage, and evidence validation.

11.6 Safety versus utilization

Prepartitioned rights can strand capacity in a quiet site while a busy site denies work. Rebalancing reduces stranding when connectivity exists but adds transfer latency and forecasting. A central coordinator can achieve high utilization under reliable connectivity and may outperform LETS. The expected advantage is not universal throughput; it is safe local progress under disconnection with a bounded and explainable authority horizon.

12 Limitations and Threats to Validity

12.1 Conceptual limitations

Novelty by composition. The contribution combines mature mechanisms. The audit found no equivalent composition, but absence from the reviewed corpus is not proof of novelty. New or differently named prior work could invalidate the claim.

Policy correctness. Conservation only enforces configured dimensions and costs. A missing effect channel or incorrectly cheap transition can cause harm without violating the formal invariant.

Complete mediation. A bypassable executor invalidates transition soundness. The current kernel does not enforce operating-system or hardware isolation.

Trusted wardens. A malicious warden can mint rights, alter state, or issue false receipts. Byzantine and threshold designs are outside the main model.

Clock dependence. The revocation-time bound depends on clock uncertainty. Residual-vector safety does not, but safe reclamation and TTL exposure do.

Evidence truth. The architecture verifies predicates and provenance hooks, not physical truth. Sensor compromise remains a domain problem.

Privacy. Ancestor paths and audit receipts may expose organizational structure and activity. Compact private membership proofs are future work.

12.2 Implementation limitations

The reference kernel is in-memory Python. It lacks durable transactions, network authentication, executor receipt replay storage, transfer garbage collection, full cross-warden active-lease migration, and crash recovery. Branch revocation is local to a warden in the prototype; partition propagation is analyzed and simulated rather than implemented end to end. Signatures are checked per action without caching. The TLA+ specification is provided but not claimed as an executed model-check result.

12.3 Experimental threats

The preliminary simulator structurally favors the intended semantics. The central baseline loses connectivity by construction; the eventual baseline is unsafe by construction; metadata values are analytic cells rather than bytes; and the microbenchmark compares unequal work. No physical sensors, wide-area network, consensus baseline, optimized bounded counter, or production storage engine are included. Statistical significance under complete distribution separation does not compensate for synthetic construct validity.

12.4 External validity

Workloads with refundable reservations, shared physical dynamics, continuous controls, stochastic risk, or effects that cannot be atomically mediated may not fit a nonnegative transition-cost vector. A high-frequency control loop may find per-transition signatures too expensive and require local hardware enforcement or batched rights. Open ecosystems without shared trusted wardens require a different trust model.

13 Future Work

The immediate work is engineering rather than feature expansion: persistent multi-warden execution, receipt-enforcing executors, transfer compaction, machine checking, competitive baselines, realistic workloads, and failure injection. After that foundation, several extensions are credible.

Verified implementation. Refine the TLA+ or another formal model to implementation traces, add timed properties, and verify an atomic storage kernel or isolate it on a verified OS such as seL4.

Byzantine or threshold wardens. Use threshold signatures and replicated state to tolerate compromised enforcement nodes. This changes latency, liveness, and transfer semantics and should be a separate contribution.

Privacy-preserving lineage. Replace explicit ancestor paths with accumulators, Merkle proofs, or anonymous credentials that support branch matching and selective disclosure.

Active-subtree migration. Implement and verify single-authority handoff with executor audience changes, timeout recovery, and no duplicated residual.

Adaptive escrow scheduling. Learn or optimize rights placement from demand while preserving the hard invariant. This could create a genuine planning or learning contribution layered above LETS.

Evidence-carrying transitions. Define typed, time-bounded evidence certificates and formally connect provenance, freshness, and HSM guards.

Replication benchmark. Build a public benchmark where spawn, recovery, migration, and mutation are ordinary lifecycle actions and evaluate containment, utility, lineage growth, and revocation exposure.

Economic accounting. Model replenishable resources, prices, reservations, refunds, and externalities. The current conservation model is intentionally monotone and nonnegative.

14 Conclusion

Autonomous populations need a systems-level boundary between reasoning and effects. Existing research already supplies nearly every primitive needed for that boundary. The remaining candidate contribution is a disciplined composition: stable wardens maintain finite escrow, dynamic agents hold attenuated nested leases, protected effects are HSM transitions with vector costs, and branch revocation is expressed as a bounded exposure rather than an impossible promise of immediate offline control.

LETS is therefore not a theory of machine intelligence and not a claim that replication itself is new. It is a proposed abstract object for population-level authority. The formal kernel preserves conservation under recursive spawn and partition; the stable-warden design separates safety metadata from historical agent identity; and finite leases expose the unavoidable revocation–availability tradeoff. The included implementation and checker establish that the idea is executable, but a top-tier paper requires a persistent distributed system, fair baselines, realistic sensor or logistics workloads, safe compaction, and stronger formal validation. The project is publishable only if those experiments show that lineage transition escrow offers more than a renamed quota tree.

A Proof Details and Counterexamples

A.1 Operation delta table

Table 3: Componentwise deltas in the conservation equation.

Operation	Warden free $\sum \boldsymbol{f}$	Lease residual $\sum \boldsymbol{r}$	Consumed $\boldsymbol{C}$	In flight $\sum \boldsymbol{x}$
Issue root $\boldsymbol{v}$	$-\boldsymbol{v}$	$+\boldsymbol{v}$	0	0
Spawn child $\boldsymbol{v}$	0	$-\boldsymbol{v}$ parent, $+\boldsymbol{v}$ child	0	0
Execute cost $\boldsymbol{c}$	0	$-\boldsymbol{c}$	$+\boldsymbol{c}$	0
Close/reclaim $\boldsymbol{r}$	$+\boldsymbol{r}$	$-\boldsymbol{r}$	0	0
Prepare transfer $\boldsymbol{v}$	$-\boldsymbol{v}$	0	0	$+\boldsymbol{v}$
Accept transfer $\boldsymbol{v}$	$+\boldsymbol{v}$ at target	0	0	$-\boldsymbol{v}$
Revoke/epoch update	0	0	0	0

A.2 Counterexample: non-idempotent transfer

Suppose source A prepares a transfer of one right to target B, reducing A from one to zero and creating one in-flight right. If B accepts the same duplicated message twice without an idempotency key, it increases its pool by two while the in-flight term decreases only once. The system now contains one extra right. Exactly-once acceptance or an equivalent deduplication invariant is therefore necessary.

A.3 Counterexample: child renewal beyond parent

A parent valid until time 10 creates a child valid until time 5. If the child can renew itself until time 100 without consulting the ancestor bound, revoking or allowing the parent to expire at time 10 does not bound descendant authority. The reference kernel includes a test that caps descendant renewal at the parent expiry.

A.4 Counterexample: HSM-free quota

A lease with one remaining action right receives a proposal to `act`. If policy requires `observe` then `validate` before `act`, a counter-only system may authorize the action because the quantity remains. An HSM-mediated system rejects it from the wrong source state. Conversely, an HSM without escrow can authorize the same legal action for an unbounded number of descendants.

B Detailed API and Protocol Fields

B.1 Lease grant fields

Field	Purpose
`lease_id`	Unique grant identifier; not sufficient by itself for replay prevention.
`lineage_id`	Root population identifier.

Field	Purpose
`parent_id`	Immediate authorization parent.
`subject_id`	Authenticated process, workload, or device identity.
`warden_id`	Current authoritative warden.
`allocation`	Original nonnegative vector allocated to the lease.
`residual`	Mutable authoritative remaining vector in durable warden state.
`capabilities`	Qualitative action classes, attenuated on spawn.
`machine_digest`	Versioned HSM specification.
`state`	Mutable authoritative HSM state.
`ancestor_proof`	Data used for branch membership and revocation matching.
`branch_epoch`	Monotonic revocation/ownership epoch.
`issued_at`, `expires_at`	Nested validity interval.
`sequence`	Monotonic mutable operation/receipt sequence.
`signature`	Warden authentication over immutable grant fields.

B.2 Receipt fields

A receipt should include: receipt identifier; warden key identifier; policy and machine versions; lease, lineage, subject, and executor identifiers; transition name and source/target states; cost vector; resulting sequence; evidence digest; request nonce; issue/expiry times; and signature. The executor should reject wrong audience, stale receipt, reused nonce, nonmonotonic sequence, or unknown key epoch.

C Reproducibility Checklist

- Source for the reference kernel, tests, benchmark, and checker is included.
- Raw preliminary CSV and JSON results are included under `results/`.
- Plot and diagram generation scripts are included under `figures/`.
- The finite TLA+ specification and checker are included under `formal/`.
- The bibliography and citation-verification table are included.
- Unit-test and checker logs are included.
- Preliminary experiments record seeds; full hardware and container provenance remains future work.
- The current artifact does not claim an executed TLC run, distributed persistence, or active-subtree migration.
- A clean full submission should add lockfiles, container digests, one-command reproduction, and immutable raw-data hashes.

D Broader Literature Map

Table 5: Representative literature map and its role in the novelty audit. The machine-readable matrix contains all 84 reviewed entries.

Area	Established basis	Consequence for LETS
Autonomous agents	Agent theory and Contract Net [71, 77]	Autonomy and delegation are background, not novelty.
Planning	HTN/SHOP2 and recursive search [26, 54, 79]	Reasoners propose transitions but do not enforce effects.
Actors and protocols	Actor model, HAPN, dynamic protocols [1, 9, 37, 78]	Dynamic creation and HSM protocols are established.
Agent runtimes	FIPA/JADE, Agent libOS, Agent Contracts [12, 29, 81, 83]	Generic lifecycle, lineage, capabilities, and budgets are non-claims.
Agent governance	Faramesh, AgentBound, Provenact, Fixed Ceiling [28, 40, 59, 84]	Complete mediation, stateful policy, and ceilings substantially narrow the claim.
Escrow and quotas	Escrow, distributed quotas, tree quotas [11, 43, 56, 61, 75]	Local rights and hierarchy are established; lineage transition semantics remain candidate synthesis.
CRDT invariants	Bounded counters, dynamic identities, exactly-once transfer [5, 10, 25, 70]	Partition-safe quantity is established; stable-warden subject separation must add value.
Finite delegation	Quota-based recursive delegation [2]	Recursive quantitative delegation is explicitly not claimed.
Capabilities	Capability semantics, Macaroons, WAVE [8, 13, 21]	Qualitative attenuation is a component, not a contribution.
Revocation	Distributed access control and delegation SoK [17, 35]	Offline revocation must be stated as bounded exposure, not immediacy.
Formal methods	Statecharts, Petri nets, TLA, model checking [18, 33, 44, 52]	Formal tools support specification; they are not novelty.
Runtime assurance	Security automata, shields, online verification [14, 58, 68]	Warden mediation is close to established runtime enforcement.
CPS and sensors	CPS and sensor-network coordination [4, 27, 36, 38, 46]	Supplies application constraints and evidence workloads.
Swarm systems	Flocking, ant systems, PSO, swarm robotics [15, 23, 41, 64, 76]	Decentralized populations are established; hard lineage authority is distinct.
Self-management	Self-stabilization, autonomic computing, Rainbow [22, 30, 42]	Recovery and adaptation must also conserve rights.
Replication	Self-reproducing automata, viruses, biological/evolutionary models [19, 24, 45, 72, 74]	Replication is one lifecycle behavior, not the central novelty.

Area	Established basis	Consequence for LETS
Modern replication risk	Agent Matrix, autonomous replication, adaptive worms, self-evolving safety [3, 31, 48, 82]	Motivates containment but does not establish LETS's object.
AI safety	Concrete problems, gridworlds, off-switch, multi-agent security tax [7, 32, 47, 57]	Evaluation must report safety–utility trade-offs.

E Candidate-Idea Audit Summary

The separate research dossier evaluates six candidates. LETS ranked first because it has a precise invariant, implementable object, reasoner independence, and a direct two-month path. Evidence-carrying transitions and lineage branch containment remain useful extensions. A graph-rewrite lifecycle compiler was rejected for excessive implementation risk; a replication benchmark was retained as an evaluation direction rather than the main contribution; and topology-adaptive sensor replication was rejected because it collided strongly with existing scheduling and autoscaling work.

The final novelty statement is:

> LETS does not introduce capabilities, leases, recursive delegation, hierarchical state machines, escrow, bounded counters, lineage tracking, or complete mediation. It proposes a lineage-oriented transition-escrow object that composes these mechanisms to preserve a multidimensional population invariant for recursively created autonomous agents during network partitions, while separating stable escrow replicas from ephemeral subjects and bounding disconnected branch-revocation exposure.

References

[1] Gul A. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems.* MIT Press, 1986. ISBN 9780262010924.

[2] Isaac Agudo, Carmen Fernandez-Gago, and Javier Lopez. Delegating privileges over finite resources: A quota based delegation approach. In *Formal Aspects in Security and Trust*, volume 5491 of *Lecture Notes in Computer Science*, pages 302–315, 2009. doi: 10.1007/978-3-642-01465-9_20.

[3] Alena Air, Reworr, Nikolaj Kotov, Dmitrii Volkov, John Steidley, and Jeffrey Ladish. Language models can autonomously hack and self-replicate, 2026.

[4] Ian F. Akyildiz, Weilian Su, Yogesh Sankarasubramaniam, and Erdal Cayirci. Wireless sensor networks: A survey. *Computer Networks*, 38(4):393–422, 2002. doi: 10.1016/S1389-1286(01)00302-4.

[5] Paulo Sérgio Almeida and Carlos Baquero. Scalable eventually consistent counters over unreliable networks. *Distributed Computing*, 32(1):69–89, 2019. doi: 10.1007/s00446-017-0322-2.

[6] Rajeev Alur and David L. Dill. A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235, 1994. doi: 10.1016/0304-3975(94)90010-8.

[7] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety, 2016.

[8] Michael P. Andersen, Sam Kumar, Moustafa AbdelBaky, Gabe Fierro, John Kolb, Hyung-Sin Kim, David E. Culler, and Raluca Ada Popa. Wave: A decentralized authorization framework with transitive delegation. In *28th USENIX Security Symposium*, pages 1375–1392. USENIX Association, 2019. ISBN 978-1-939133-06-9.

[9] Alexander Artikis. Dynamic protocols for open agent systems. In *Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems*, pages 97–104, 2009.

[10] Valter Balegas, Diogo Serra, Sergio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno Preguiça. Extending eventually consistent cloud databases for enforcing numeric invariants. In *2015 IEEE 34th Symposium on Reliable Distributed Systems*, pages 31–36, 2015. doi: 10.1109/SRDS.2015.32.

[11] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. Dqmp: A decentralized protocol to enforce global quotas in cloud environments. In *Stabilization, Safety, and Security of Distributed Systems*, volume 7596 of *Lecture Notes in Computer Science*, pages 217–231, 2012. doi: 10.1007/978-3-642-33536-5_21.

[12] Fabio Bellifemine, Agostino Poggi, and Giovanni Rimassa. Jade: A fipa2000 compliant agent development environment. In *Proceedings of the Fifth International Conference on Autonomous Agents*, pages 216–217, 2001. doi: 10.1145/375735.376120.

[13] Arnar Birgisson, Joe Gibbs Politz, Ulfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium*, 2014. doi: 10.14722/ndss.2014.23212.

[14] Roderick Bloem, Bettina Könighofer, Robert Könighofer, and Chao Wang. Shield synthesis: Runtime enforcement for reactive systems. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 9035 of *Lecture Notes in Computer Science*, pages 533–548, 2015. doi: 10.1007/978-3-662-46681-0_51.

[15] Manuele Brambilla, Eliseo Ferrante, Mauro Birattari, and Marco Dorigo. Swarm robotics: A review from the swarm engineering perspective. *Swarm Intelligence*, 7(1):1–41, 2013. doi: 10.1007/s11721-012-0075-2.

[16] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

[17] Laurent Chuat, AbdelRahman Abdou, Ralf Sasse, Christoph Sprenger, David Basin, and Adrian Perrig. Sok: Delegation and revocation, the missing links in the web's chain of trust. In *2020 IEEE European Symposium on Security and Privacy*, pages 624–638, 2020. doi: 10.1109/EuroSP48549.2020.00046.

[18] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.

[19] Fred Cohen. Computer viruses: Theory and experiments. *Computers & Security*, 6(1):22–35, 1987. doi: 10.1016/0167-4048(87)90122-2.

[20] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-2636.

[21] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.

[22] Edsger W. Dijkstra. Self-stabilizing systems in spite of distributed control. *Communications of the ACM*, 17(11):643–644, 1974. doi: 10.1145/361179.361202.

[23] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1): 29–41, 1996. doi: 10.1109/3477.484436.

[24] Manfred Eigen. Selforganization of matter and the evolution of biological macromolecules. *Naturwissenschaften*, 58:465–523, 1971. doi: 10.1007/BF00623322.

[25] Vitor Enes, Carlos Baquero, Paulo Sérgio Almeida, and Jo ao Leitão. Borrowing an identity for a distributed counter. In *Proceedings of the 3rd International Workshop on Principles and Practice of Consistency for Distributed Data*, pages 4:1–4:3, 2017. doi: 10.1145/3064889.3064894.

[26] Kutluhan Erol, James Hendler, and Dana S. Nau. Htn planning: Complexity and expressivity. In *Proceedings of the Twelfth National Conference on Artificial Intelligence*, pages 1123–1128, 1994.

[27] Deborah Estrin, Ramesh Govindan, John Heidemann, and Satish Kumar. Next century challenges: Scalable coordination in sensor networks. In *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking*, pages 263–270, 1999. doi: 10.1145/313451.313556.

[28] Amjad Fatmi. Faramesh: A protocol-agnostic execution control plane for autonomous agent systems, 2026.

[29] Foundation for Intelligent Physical Agents. Fipa agent management specification. Technical Report SC00023K, FIPA, 2004.

[30] David Garlan, Shang-Wen Cheng, An-Cheng Huang, Bradley Schmerl, and Peter Steenkiste. Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer*, 37(10): 46–54, 2004. doi: 10.1109/MC.2004.175.

[31] Jonas Guan, Tom Blanchard, Hanna Foerster, Hengrui Jia, Gabriel Huang, and Nicolas Papernot. Ai agents enable adaptive computer worms, 2026.

[32] Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, and Stuart Russell. The off-switch game. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pages 220–227, 2017. doi: 10.24963/ijcai.2017/32.

[33] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987. doi: 10.1016/0167-6423(87)90035-9.

[34] David Harel and Amnon Naamad. The statemate semantics of statecharts. *ACM Transactions on Software Engineering and Methodology*, 5(4):293–333, 1996. doi: 10.1145/235321.235322.

[35] Richard J. Hayton, Jean M. Bacon, and Ken Moody. Access control in an open distributed environment. In *1998 IEEE Symposium on Security and Privacy*, pages 3–14, 1998. doi: 10.1109/SECPRI.1998.674819.

[36] Wendi Rabiner Heinzelman, Anantha Chandrakasan, and Hari Balakrishnan. Energy-efficient communication protocol for wireless microsensor networks. In *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, page 10 pp., 2000. doi: 10.1109/HICSS.2000.926982.

[37] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, pages 235–245, 1973.

[38] Jason Hill, Robert Szewczyk, Alec Woo, Seth Hollar, David Culler, and Kristofer Pister. System architecture directions for networked sensors. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 93–104, 2000. doi: 10.1145/356989.356998.

[39] Roland R. Rodriguez Jr. Agent identity uri scheme: Topology-independent naming and capability-based discovery for multi-agent systems, 2026.

[40] Anuj Kaul, Qianlong Lan, and Pranay Gupta. Agentbound: Verifiable behavioral governance for autonomous ai agents, 2026.

[41] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 – International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.

[42] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1): 41–50, 2003. doi: 10.1109/MC.2003.1160055.

[43] Ewnetu Bayuh Lakew, Lei Xu, Francisco Hernández-Rodríguez, Erik Elmroth, and Claus Pahl. A tree-based protocol for enforcing quotas in clouds. In *2014 IEEE World Congress on Services*, pages 279–286, 2014. doi: 10.1109/SERVICES.2014.57.

[44] Leslie Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994. doi: 10.1145/177492.177726.

[45] Christopher G. Langton. Self-reproduction in cellular automata. *Physica D: Nonlinear Phenomena*, 10(1–2):135–144, 1984. doi: 10.1016/0167-2789(84)90256-2.

[46] Edward A. Lee. Cyber physical systems: Design challenges. In *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, pages 363–369, 2008. doi: 10.1109/ISORC.2008.25.

[47] Jan Leike, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. Ai safety gridworlds, 2017.

[48] Ruixiao Lin, Xinhao Deng, Qingming Li, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhenyuan Li, Yechao Zhang, Shiwen Cui, Changhua Meng, Tianwei Zhang, Xingjun Ma, Qi Li, Ke Xu, and Shouling Ji. Safety in self-evolving llm agent systems: Threats, amplification, and case studies, 2026.

[49] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2308.03688.

[50] Sumana Malkapuram, Sameera Gangavarapu, Kailashnath Reddy Kavalakuntla, and Ananya Gangavarapu. Context lineage assurance for non-human identities in critical multi-agent systems, 2025.

[51] Robin Milner. Bigraphical reactive systems. In *CONCUR 2001 – Concurrency Theory*, volume 2154 of *Lecture Notes in Computer Science*, pages 16–35, 2001. doi: 10.1007/3-540-44685-0_2.

[52] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77 (4):541–580, 1989. doi: 10.1109/5.24143.

[53] Toby C. Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. sel4: From general purpose to a proof of information flow enforcement. In *2013 IEEE Symposium on Security and Privacy*, pages 415–429, 2013. doi: 10.1109/SP.2013.35.

[54] Dana Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. Shop2: An htn planning system. *Journal of Artificial Intelligence Research*, 20: 379–404, 2003. doi: 10.1613/jair.1141.

[55] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997. doi: 10.1145/263699.263712.

[56] Patrick E. O'Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4):405–430, 1986. doi: 10.1145/7239.7265.

[57] Pierre Peigné, Mikolaj Kniejski, Filip Sondej, Matthieu David, Jason Hoelscher-Obermaier, Christian Schroeder de Witt, and Esben Kran. Multi-agent security tax: Trading off security and collaboration capabilities in multi-agent systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(26):27573–27581, 2025. doi: 10.1609/aaai.v39i26.34970.

[58] Christian Pek, Stefanie Manzinger, Markus Koschi, and Matthias Althoff. Using online verification to prevent autonomous vehicles from causing accidents. *Nature Machine Intelligence*, 2(9):518–528, 2020. doi: 10.1038/s42256-020-0225-y.

[59] Yuxiang Peng and Xiaodi Wu. Stateful governance for concurrent agentic systems, 2026.

[60] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science*, pages 46–57, 1977. doi: 10.1109/SFCS.1977.32.

[61] Kristal T. Pollack, Darrell D. E. Long, Richard A. Golding, Ralph A. Becker-Szendy, and Benjamin Reed. Quota enforcement for high-performance distributed storage systems. In *24th IEEE Conference on Mass Storage Systems and Technologies*, pages 72–86, 2007. doi: 10.1109/MSST.2007.4367965.

[62] Wenjie Qu, Xuandong Zhao, Jiaheng Zhang, and Dawn Song. Self-sovereign agent, 2026.

[63] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4295–4304, 2018. doi: 10.48550/arXiv.1803.11485.

[64] Craig W. Reynolds. Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, pages 25–34, 1987. doi: 10.1145/37401.37406.

[65] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.

[66] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551, 2023. doi: 10.48550/arXiv.2302.04761.

[67] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers, 2003.

[68] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. doi: 10.1145/353323.353382.

[69] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2303.11366.

[70] Ali Shoker, Paulo Sérgio Almeida, and Carlos Baquero. Exactly-once quantity transfer. In *2015 IEEE 34th Symposium on Reliable Distributed Systems Workshops*, pages 68–73, 2015. doi: 10.1109/SRDSW.2015.10.

[71] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.

[72] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002. doi: 10.1162/106365602320169811.

[73] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, pages 18:1–18:17, 2015. doi: 10.1145/2741948.2741964.

[74] John von Neumann. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966. Edited and completed by Arthur W. Burks.

[75] Michael Walfish, J. D. Zamfirescu, Hari Balakrishnan, David Karger, and Scott Shenker. Distributed quota enforcement for spam control. In *3rd USENIX Symposium on Networked Systems Design and Implementation*, pages 281–296. USENIX Association, 2006.

[76] Justin Werfel, Kirstin Petersen, and Radhika Nagpal. Designing collective behavior in a termite-inspired robot construction team. *Science*, 343(6172):754–758, 2014. doi: 10.1126/science.1245842.

[77] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995. doi: 10.1017/S0269888900008122.

[78] Nitin Yadav, Lin Padgham, and Michael Winikoff. A tool for defining agent protocols in hapn (demonstration). In *Proceedings of the 14th International Conference on Autonomous Agents and Multiagent Systems*, pages 1935–1936, 2015.

[79] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2305.10601.

[80] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.03629.

[81] Qing Ye and Jing Tan. Agent contracts: A formal framework for resource-bounded autonomous ai systems, 2026.

[82] Boxuan Zhang, Yi Yu, Jiaxuan Guo, and Jing Shao. Dive into the agent matrix: A realistic evaluation of self-replication risk in llm agents, 2025.

[83] Yingqi Zhang. Agent libos: A library-os-inspired runtime for long-running, capability-controlled llm agents, 2026.

[84] Zhaoxi Zhang and Xiaomei Zhang. Are you still the agent i authorized? earned authority under a fixed ceiling for evolving agents, 2026.

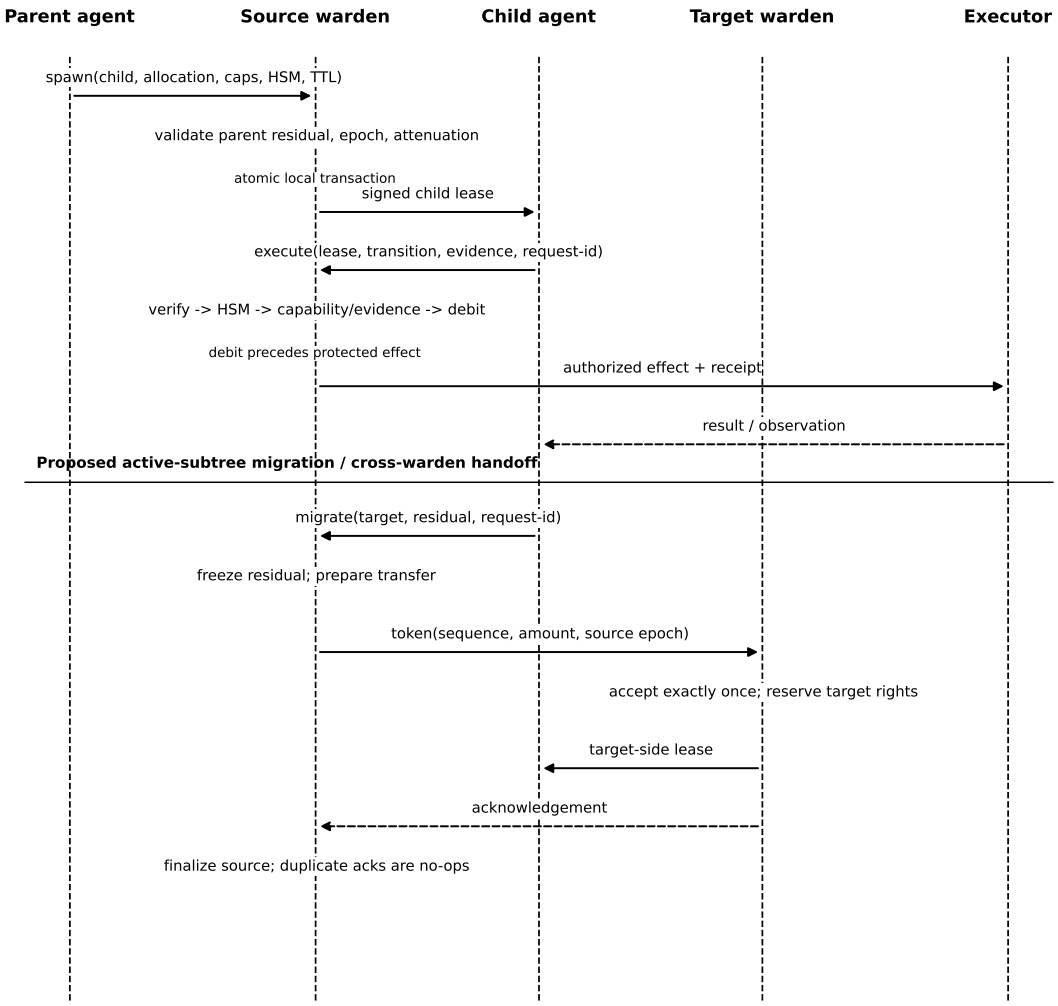


Figure 3: Spawn and protected execution remain local while residual rights exist. The lower migration path is proposed and not implemented in the v0 kernel.

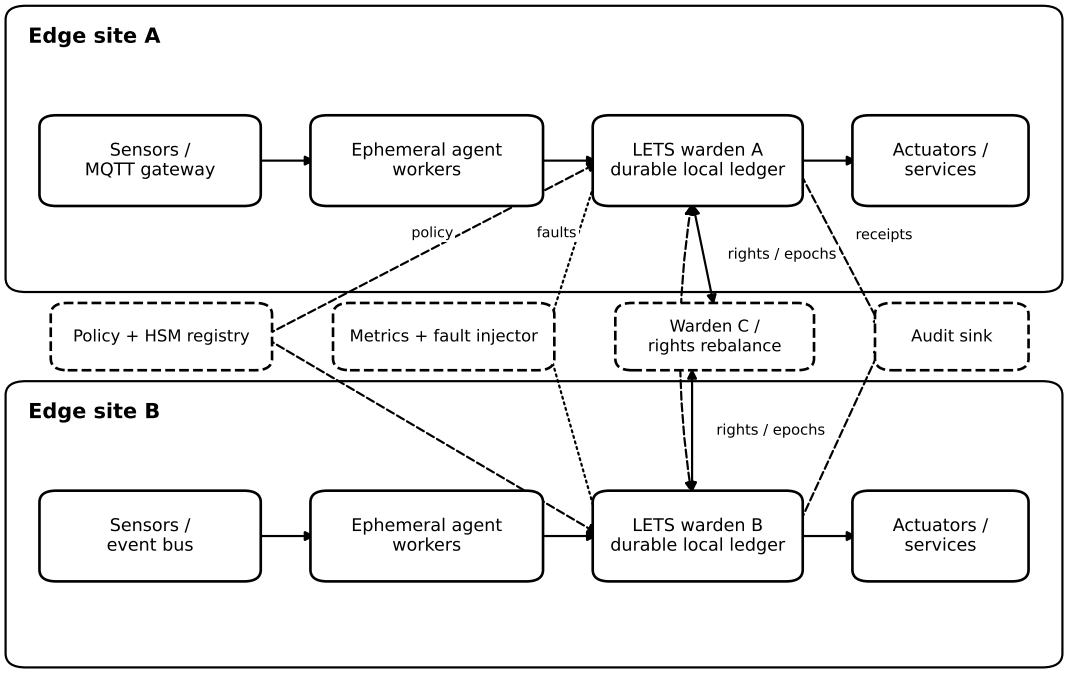


Figure 4: Proposed evaluation deployment. Wardens are stable infrastructure; agent workers can churn. MQTT or another event bus supplies sensor evidence.

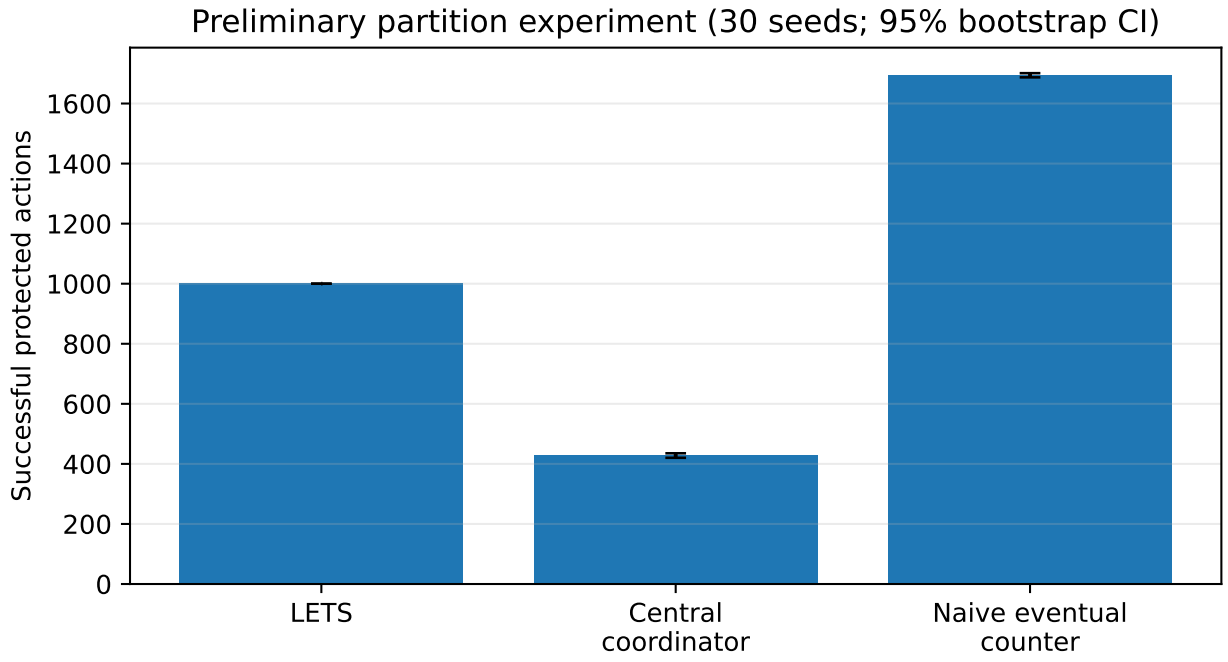


Figure 5: Preliminary successful protected actions. The naive eventual counter reports more work because it exceeds the configured budget; useful work must be interpreted with Figure 6.

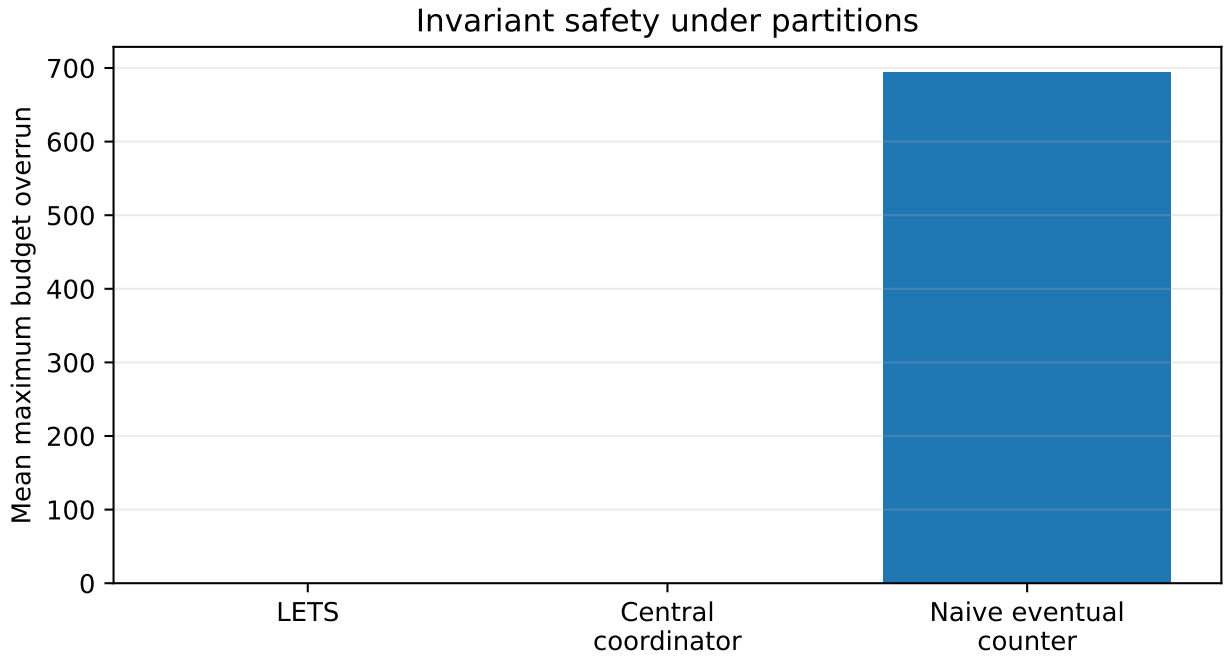
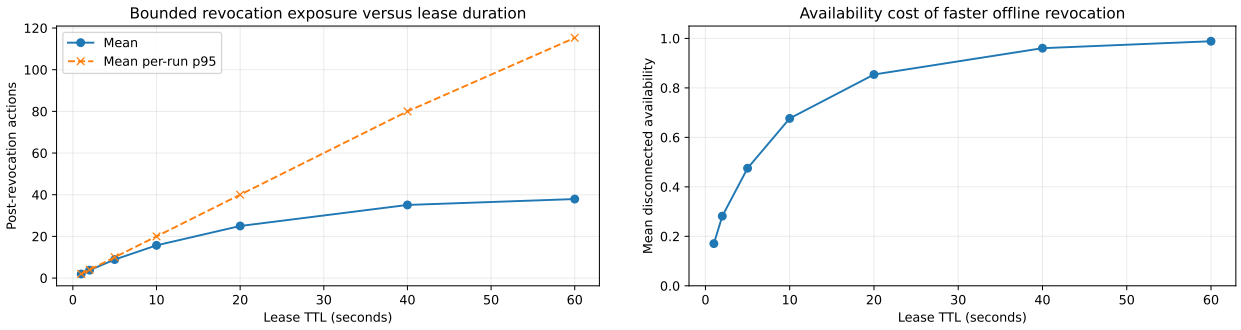


Figure 6: Preliminary maximum budget overrun. LETS and the central coordinator are safe in this simulator; the naive eventual counter violates the bound in all 30 runs.



(a) Post-revocation actions.

(b) Disconnected availability.

Figure 7: Preliminary lease-duration trade-off. Short leases limit offline exposure but reject more legitimate disconnected work.

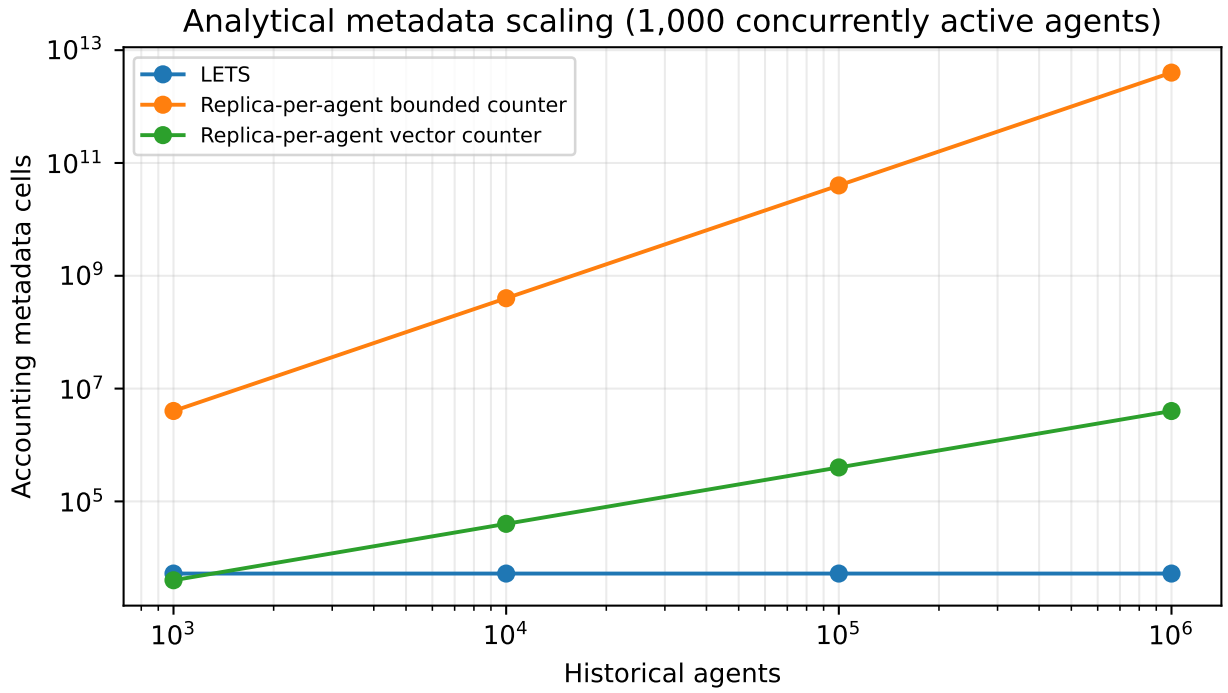


Figure 8: Analytic online-metadata comparison. This is a structural model, not measured bytes. A complete implementation must demonstrate safe compaction and report archival audit growth separately.